📚 Workflow Agentic - Upstream Prevention & Detailed Diagnostics

Repository: github.com/sulastambunan/Workflow-Agentic-Sulas
PR: #5 (docs/upstream-prevention-diagnostics)
Date: 2026-01-15
Status: Ready for Review & Merge

📄 Daftar Isi

- Overview
- Files Created
- Core Principles
- Integration & Flow
- Checklist Implementation
- Workflow Phases
- Updates to Existing Files
- Next Steps

🎯 Overview

Dokumentasi baru ini fokus pada **upstream prevention** (prevent bugs saat grounding/design) dan **detailed diagnostics** (jelas bagaimana handle kalau test fail).

Problem yang Diselesaikan

- ❌ Tautology hanya detect di PR review → ✅ NOW prevent di grounding
- ❌ Swallowed errors hanya detect di PR review → ✅ NOW prevent di implementation
- ❌ Hardcoded values hanya detect di PR review → ✅ NOW prevent di design
- ❌ Test failure unclear saat runtime → ✅ NOW detailed diagnosis framework
- ❌ Automation tidak traceable ke TC → ✅ NOW centralized verification checklist
- ❌ Environment issues surprise saat verify → ✅ NOW ask Phase 0

📁 Files Created (7)

1. 14-executability-principle.md

Rule: EVERYTHING must be executable in dev/stg

- Covers all layers (SPEC, USECASE, Layer3, FACTORY, PRE)
- Environment parity requirements
- Checklists per phase (grounding, design, implement, verify)
- Anti-patterns & what NOT to do

2. 15-test-failure-reporting.md ✅ CREATED

Standard failure report format with decision tree

- 5 failure types (PRE, FACTORY, USECASE, Layer3, SPEC)
- Root cause analysis per type
- Evidence requirements (screenshot, video, logs, stack trace)
- Decision tree: "What to do when test fails?"

3. 16-assertion-design.md

Prevent tautology, swallowed errors, hardcoded values

- Rule 1: Every assertion must be able to FAIL
- Rule 2: Every error must PROPAGATE
- Rule 3: Every value from TC (not code)
- Error handling patterns per layer
- Assertion design template BEFORE implementation

4. 17-traceability-record.md

Centralized TC verification

- AUTOMATION-CHECKLIST-<TC-ID>.md template
- Given/When/Then mapping
- Red flag prevention checklist
- Self-review before push

5. INTEGRATION-MAP.md

How new docs integrate with existing workflow

- 3-layer review system: static code → executability → runtime
- Integration with skill automation-pr-review
- Where to add executability checks in skill

6. README-NEW-DOCUMENTATION.md

Master README summarizing all changes

- Overview & core principles
- Implementation plan
- Reading order for different roles
- Expected outcomes

7. UPDATES-TO-EXISTING-FILES.md

Manual merge guide for 5 existing files

- Detailed what/where/how for each update
- Cross-reference strategy
- How to apply changes

💡 Core Principles

Principle 1: Executability (doc 14)

NOT: "Assume code will work"
BUT: "Verify everything works in REAL environment"

Every assertion, endpoint, selector, value, precondition
must be VERIFIED IN REAL ENVIRONMENT before code.

Principle 2: Assertion Quality (doc 16)

Rule 1: Every assertion must be able to FAIL
✅ expect(status).toBe('DRAFT')
❌ expect(true).toBe(true)

Rule 2: Every error must PROPAGATE
✅ throw new Error(`Failed: ${msg}`)
❌ .catch(() => false)

Rule 3: Every value from TC (not code)
✅ From TC Then clause
❌ From code assumption

Principle 3: Detailed Failure Diagnosis (doc 15)

```
When test FAILS at runtime:
  1. Identify LAYER (PRE, FACTORY, USECASE, Layer3, SPEC)
  2. Root cause analysis
  3. Evidence (screenshot, video, logs, stack trace)
  4. Decision tree (fix code? re-ground? escalate?)
```

```
NOT: "Test failed"
BUT: "SPEC assertion fail at line 26..."
```

Principle 4: Centralized Traceability (doc 17)

NOT: Comments scattered in code
BUT: AUTOMATION-CHECKLIST-<TC-ID>.md

✅ Explicit: TC clause → assertion mapping
✅ Self-reviewed: engineer verify match
✅ Centralized: one place to see full coverage
✅ Plus: Code comments as backup

🔗 Integration & Flow

3-Layer Review System

Layer	When	What	Tool
Layer 1: Static Code	PR opened	Clause table, red flags, pitfalls, gates, DDD	skill automation-pr-review (existing)
Layer 2: Executability	Grounding + Code	All layers executable? Selector exist? Endpoint respond?	doc 14-executability-principle (NEW)
Layer 3: Runtime Diagnosis	Test fails	Which layer? Root cause? Evidence? Action?	doc 15-test-failure-reporting (NEW)

Workflow Integration

```
Phase 0: INTAKE
  └─ workflow/00-ledger-intake.md + Step 6
     (ask environment, test data, account)

Phase 1: GROUNDING
  └─ doc 14-executability-principle.md
     (selector, endpoint, value ground)

Phase 2: DESIGN
  └─ doc 16-assertion-design.md
     (design assertion template, error handling)

Phase 3: MAPPING
  └─ DDD per layer, Layer3 mapping

Phase 4: IMPLEMENT
  └─ doc 14, 16 checklist
  └─ doc 17 AUTOMATION-CHECKLIST
  └─ workflow/04-implement.md + Implementation Output

Phase 5: VERIFY
  └─ Run test
  └─ IF fail: doc 15-test-failure-reporting.md
  └─ workflow/05-verify-evidence.md + Failure Diagnosis

Phase 6: PRE-PUSH
  └─ doc 17 Traceability Check
  └─ docs/shared/11-pre-push-review.md + verification

PR REVIEW (lead)
  └─ skill automation-pr-review (updated)
  └─ doc 14 executability findings
  └─ doc 15 failure diagnosis (IF tests ran + failed)
```

✅ Checklist Implementation

Phase 0: INTAKE

Add to: workflow/00-ledger-intake.md → Step 6

- ☑ Target environment (dev/stg/both)?
- ☑ Test account (role, email, credential source)?
- ☑ Test data (available, setup method)?
- ☑ Environment config (endpoint URL, feature flag, auth)?
- ☑ Data cleanup (delete/keep, unique per run)?

Phase 4: IMPLEMENT

Add to: workflow/04-implement.md → Implementation Output

- ☑ AUTOMATION-CHECKLIST-<TC-ID>.md created
- ☑ Code comments added (@covers, // src:, // expect:)
- ☑ Test PASSED (not skipped, not failed)
- ☑ PRE executed successfully
- ☑ Evidence collected (video, logs)

Phase 6: PRE-PUSH

Add to: docs/shared/11-pre-push-review.md → Traceability Verification

- ☑ AUTOMATION-CHECKLIST-<TC-ID>.md exist?
- ☑ All TC clauses covered (Given/When/Then/And)?
- ☑ Clause table complete (no missing assertions)?
- ☑ Red flags scanned (tautology, swallowed error, hardcoded)?
- ☑ Code comments present (@covers, // expect:, // src:)?
- ☑ Test result: PASSED (not skipped, not failed)?

PR REVIEW: Executability Check

Add to: skills/automation-pr-review/SKILL.md → Section 5.1 (after "5. Code checks")

- ☑ SPEC: Expected value from TC? Tautology? Can fail?
- ☑ USECASE: Calls real endpoint? Error propagates?
- ☑ Layer 3: Selector grounded (// src:)? Not hardcoded?
- ☑ FACTORY: Data varied? No API call? Schema valid?
- ☑ PRE: Endpoint verified? Response validated?

PR REVIEW: Failure Diagnosis (if tests executed)

Add to: skills/automation-pr-review/SKILL.md → Section 6.1 (after "6. Gates")

- ☑ If test failed: Layer identified?
- ☑ Root cause analysis done?
- ☑ Evidence collected (screenshot, video, logs)?
- ☑ Decision clear (fix code? re-ground? escalate?)?

📊 Workflow Phases: Before vs After

BEFORE (Downstream Detection)

```
Code Written
  ↓
Test Run
  ↓
PR Review (FIRST TIME detect: tautology, swallowed error, hardcoded)
  ↓ REQUEST_CHANGES: Rework needed
Fix Needed
  ↓
Re-test
  ↓
EXPENSIVE CYCLE
```

AFTER (Upstream Prevention + Detailed Diagnosis)

```
Phase 0: INTAKE
  └─ Ask: environment, test data, account

Phase 1: GROUNDING
  └─ Verify: selector exist, endpoint respond, value accessible

Phase 2: DESIGN
  └─ Design: assertion template, error handling plan

Phase 3: MAPPING
  └─ Map: Layer responsibility

Phase 4: IMPLEMENT
  └─ Prevent: tautology, swallowed error, hardcoded
  └─ Create: AUTOMATION-CHECKLIST-<TC-ID>.md

Phase 5: VERIFY
  └─ Run: test PASSED
  └─ IF fail: detailed diagnosis (layer, root cause, decision)

Phase 6: PRE-PUSH
  └─ Verify: traceability checklist

PR REVIEW
  └─ Confirm: quality at source, rarely find new issue

RESULT: Quality upstream, clear diagnosis if fail
```

Comparison

Aspect	Before	After
Executability	Assumed	Verified per phase
Assertion Design	Trial and error	Template before code
Error Handling	Surprise at runtime	Planned per layer
Failure Diagnosis	Vague error message	Structured report
TC Traceability	Scattered in code	Centralized checklist
Environment Setup	Ask Phase 5	Ask Phase 0 intake
Red Flag Detection	PR review	Upstream prevention

⚠️ Updates to Existing Files (Manual Merge Needed)

5 files need updates after PR merge:

1. workflow/00-ledger-intake.md

Add: Step 6 — Environment & Data Setup questionnaire

After: Current Step 5, before "Output" section

Benefit: Prevent environment surprises in Phase 5

2. workflow/04-implement.md

Add: Implementation Output section (at end)

Content: AUTOMATION-CHECKLIST creation, code comments, test output

Benefit: Ensure upstream prevention, not downstream detection

3. workflow/05-verify-evidence.md

Add: Failure Diagnosis section (after "Reporting a result")

Content: Identify layer, root cause, evidence, decision tree

Benefit: Clear failure diagnosis, not vague error messages

4. docs/shared/11-pre-push-review.md

Add: Traceability Verification checklist

Content: Verify AUTOMATION-CHECKLIST exist, clauses covered, red flags scanned

Benefit: Ensure traceability before PR

5. skills/automation-pr-review/SKILL.md

Add Three Sections:

- **5.1 Executability validation** (code-only check)
- **6.1 When tests executed** (PRE/USECASE/Layer3/FACTORY checks)
- **Update Section 8** (add executability & failure findings to report)

Benefit: Skill aligned with new executability principle & failure diagnosis

> **See:** UPDATES-TO-EXISTING-FILES.md (in PR) for detailed merge guide with exact locations and content

🚀 Next Steps

Immediate (After PR Created)

1. **Review PR #5:** github.com/sulastambunan/Workflow-Agentic-Sulas/pull/5
2. **Read README-NEW-DOCUMENTATION.md:** In PR, overview of all changes
3. **Check INTEGRATION-MAP.md:** How everything connects

After PR Merge

1. **Update 5 existing files** (per UPDATES-TO-EXISTING-FILES.md)
2. **Test cross-references:** All links work correctly
3. **Train team:** Use new docs in actual automation work

For Automation Engineers

1. Read: 14-executability-principle.md (what's executable)
2. Read: 16-assertion-design.md (design before code)
3. Read: 17-traceability-record.md (create AUTOMATION-CHECKLIST)
4. Reference: 15-test-failure-reporting.md (IF test fails)

For QA Lead/Reviewer

1. Read: INTEGRATION-MAP.md (3-layer review system)
2. Read: skill automation-pr-review updates (executability + failure sections)
3. Reference: 14-executability-principle.md (what to look for)
4. Reference: 15-test-failure-reporting.md (failure diagnosis)

For Team Onboarding

1. Start: README-NEW-DOCUMENTATION.md (master overview)
2. Then: Core principles (14, 16, 17, 15)
3. Practice: Create AUTOMATION-CHECKLIST per TC
4. Verify: Self-review checklist before push

Success Metrics (Expected Outcomes)

> ✅ **Upstream prevention:** Tautology prevented in grounding/design
> ✅ **Detailed diagnostics:** Test failure clear (layer identified, root cause known)
> ✅ **Centralized verification:** AUTOMATION-CHECKLIST per TC
> ✅ **Faster cycle:** Less rework, faster debugging
> ✅ **Confident deployment:** Traceable to TC, quality gates passed

---

📚 **Workflow Agentic Documentation** | PR #5 | 2026-01-15

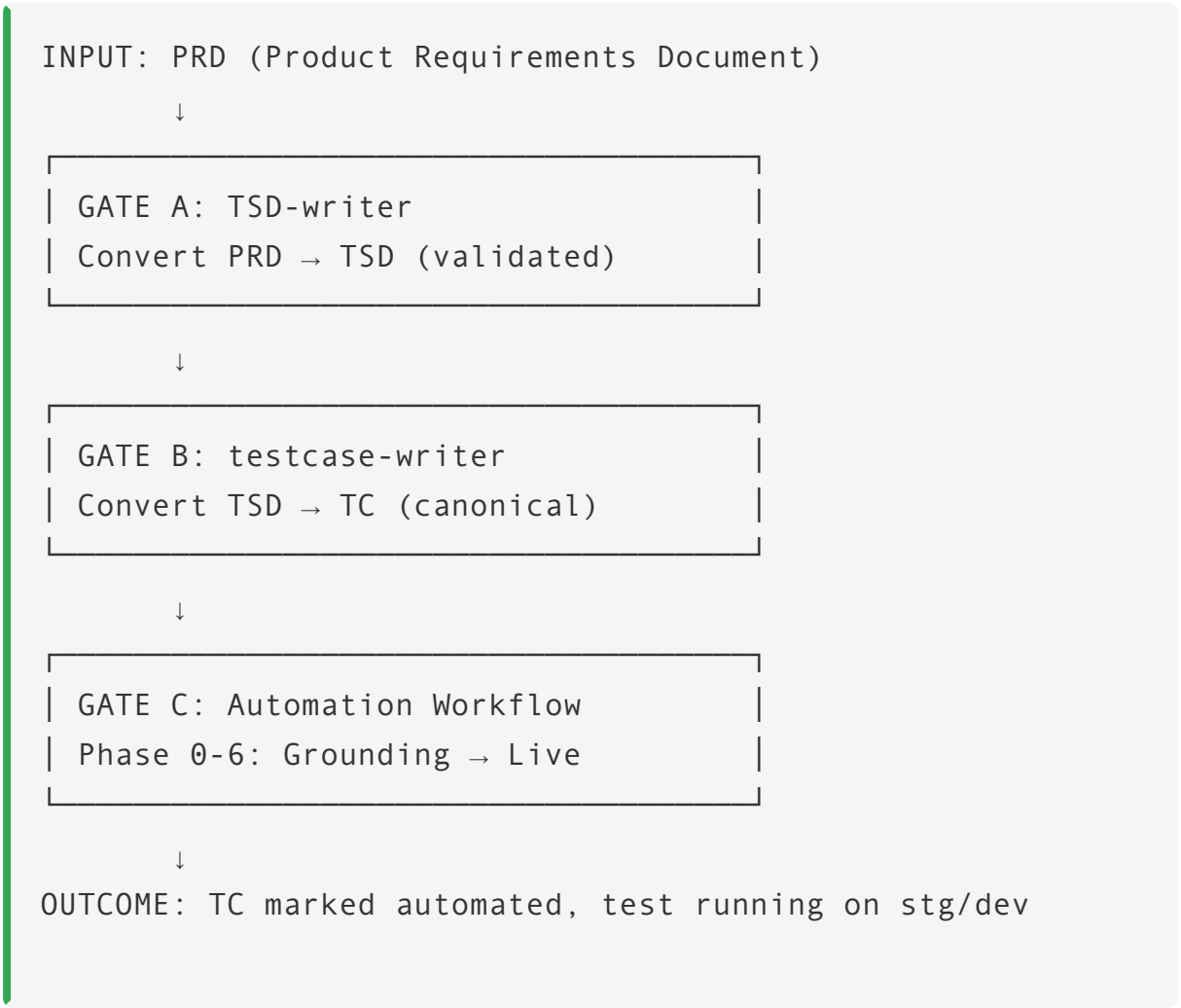
Repository: github.com/sulastambunan/Workflow-Agentic-Sulas

Generated with Claude Code | 🤖 Ready to print as PDF

End-to-End Flow: PRD → Automation Live

Goal: Complete flow from PRD to automated test running on stg/dev with upstream prevention at each gate.

3 GATES + 6 PHASES



GATE A: TSD-writer (PRD → TSD)

- **Input:** PRD document
- **Output:** TSD file (validated) + self-check results
- **Prevention:** Requirement clarity, scope, PRE identified

GATE B: testcase-writer (TSD → TC)

- **Input:** Validated TSD
- **Output:** TC files + PRE files
- **Prevention:** Data mapping, executability verified

GATE C: Automation Workflow (Phase 0-6)

Phase 0: Intake

Read TC, ask environment questions, verify automation status

Phase 1: Grounding

Ground PRE, selectors, API endpoints, data sources with file:line citations

Phase 2: Design

Design assertions per Then clause, plan DDD architecture, error handling

Phase 3: Mapping

Map to SPEC/USECASE/Layer3/FACTORY files and methods

Phase 4: Implement

Code per DDD layer, add comments, create AUTOMATION-CHECKLIST, run test

Phase 5: Verify

Run test, verify PRE executed, collect evidence, check checklist complete

Phase 6: Review & Push

Self-review, create PR, reviewer checks 3 layers, merge

Quality Gates (Upstream Prevention)

Gate/Phase	Input	Output	Key Prevention
GATE A	PRD	TSD	Requirement clarity
GATE B	TSD	TC + PRE	Data mapping
Phase 0	TC	Intake	Environment early
Phase 1	Intake	Grounding	Executability verified
Phase 4	Design	Code	Red flags prevented
Phase 6	Evidence	PR review	Upstream quality confirmed